



**Università  
di Brescia**

DIPARTIMENTO DI INGEGNERIA  
DELL'INFORMAZIONE

*Corso di Laurea*  
*in INGEGNERIA INFORMATICA*

Relazione Finale  
**Calcolo dell'Equilibrio di Nash**  
**in una rete di traffico con due popolazioni**

**Relatore:**  
**Prof. Rinaldo M. Colombo**

**Laureando:**  
**Borra Giacomo**  
**Matricola**  
**n.746260**

*Anno Accademico 2025/2026*

# Indice

|                                            |           |
|--------------------------------------------|-----------|
| <b>Introduzione</b>                        | <b>2</b>  |
| <b>1 Modello</b>                           | <b>3</b>  |
| 1.1 Descrizione del modello . . . . .      | 3         |
| 1.2 Equilibrio di Nash . . . . .           | 6         |
| 1.3 Esempi . . . . .                       | 9         |
| 1.3.1 Inizializzazione Programma . . . . . | 9         |
| 1.3.2 Esempio 1 . . . . .                  | 11        |
| 1.3.3 Esempio 2 . . . . .                  | 20        |
| <b>2 Programma</b>                         | <b>30</b> |
| 2.1 Contratto AbstractParam . . . . .      | 30        |
| 2.2 Utility . . . . .                      | 38        |
| 2.3 Classe Initializer . . . . .           | 40        |
| 2.4 Classe Computer . . . . .              | 46        |

Contributo?

# Introduzione

Nash, Reti di traffico, Braess

# 1 Modello

## 1.1 Descrizione del modello

Rappresentiamo ~~il modello di~~ una rete stradale  $\mathcal{N}$  come un grafo avente per archi le  $N$  strade  $r_1, \dots, r_N$ , le cui proprietà verranno descritte in seguito. Le due popolazioni presenti nella suddetta rete sono la popolazione *hat* e la popolazione *check*. Entrambe hanno un'origine e una destinazione, rappresentate dalle coppie di intersezioni di archi  $(\hat{\mathcal{O}}, \hat{\mathcal{D}})$  e  $(\check{\mathcal{O}}, \check{\mathcal{D}})$ , non per forza distinte. Introduciamo infine il concetto di percorso, ovvero una tupla composta da  $m$  strade adiacenti e distinte a coppie  $\gamma \equiv (r_{h_1}, \dots, r_{h_m})$ , tale che il punto finale di  $r_{h_l}$  è il punto iniziale di  $r_{h_{l+1}}$ , con  $l \in (1, \dots, m-1)$ .

Viene utile introdurre, come in [3, Cap. 2], una suddivisione di  $\mathcal{N}$  in due sottoreti,  $\hat{\mathcal{N}}$ , costituita dai percorsi  $\hat{\gamma}_1, \dots, \hat{\gamma}_{\hat{n}}$  che collegano  $\hat{\mathcal{O}}$  a  $\hat{\mathcal{D}}$ , e  $\check{\mathcal{N}}$ , costituita dai percorsi  $\check{\gamma}_1, \dots, \check{\gamma}_{\check{n}}$  che collegano  $\check{\mathcal{O}}$  a  $\check{\mathcal{D}}$ . Queste due sottoreti devono essere grafi orientati aciclici (DAG) connessi. Si noti che non è necessario imporre queste condizioni all'intera rete  $\mathcal{N}$ , anche se, per ottenere risultati rilevanti, in seguito si troveranno solo reti di traffico che sono DAG connessi.

Costruiamo ora la matrice  $\hat{\Gamma}$ , di dimensioni  $N \times \hat{n}$ , e la matrice  $\check{\Gamma}$ , di dimensioni  $N \times \check{n}$ :

$$\hat{\Gamma}_{hi} := \begin{cases} 1 & \text{se la strada } r_h \text{ appartiene al percorso } \hat{\gamma}_i \\ 0 & \text{altrimenti} \end{cases} \quad \text{con } h \in \{1, \dots, N\} \text{ e } i \in \{1, \dots, \hat{n}\};$$

$$\check{\Gamma}_{hi} := \begin{cases} 1 & \text{se la strada } r_h \text{ appartiene al percorso } \check{\gamma}_i \\ 0 & \text{altrimenti} \end{cases} \quad \text{con } h \in \{1, \dots, N\} \text{ e } i \in \{1, \dots, \check{n}\}.$$

Consideriamo la totalità di una popolazione presente sulla rete essere 1. Chiamiamo

$\hat{\theta}_i$  il numero di viaggiatori che scelgono di percorrere  $\hat{\gamma}_i$ , con  $i \in \{1, \dots, \hat{n}\}$ , e  $\check{\theta}_j$  il numero di viaggiatori che scelgono di percorrere  $\check{\gamma}_j$ , con  $j \in \{1, \dots, \check{n}\}$ . Allora  $\hat{\theta} \equiv (\hat{\theta}_1, \dots, \hat{\theta}_{\hat{n}})$  è un punto del semplice  $S^{\hat{n}}$  di dimensione  $\hat{n}$ , dove  $\hat{\theta}_i \in [0, 1]$ ; rispettivamente  $\check{\theta} \equiv (\check{\theta}_1, \dots, \check{\theta}_{\check{n}})$  è un punto del semplice  $S^{\check{n}}$  di dimensione  $\check{n}$ , con  $\check{\theta}_j \in [0, 1]$ .  $\hat{\theta}$  e  $\check{\theta}$  sono identificabili come vettori colonna.

Definiamo i vettori  $\hat{\nu}$  e  $\check{\nu}$ , le cui componenti, rispettivamente  $\hat{\nu}_i$  e  $\check{\nu}_i$  con  $i = 1, \dots, N$ , rappresentano il numero di veicoli (delle popolazioni *hat* e *check*) sulla strada  $r_i$ . Sappiamo dunque che:

$$\begin{aligned}\hat{\nu} &= \hat{\Gamma} \hat{\theta} \\ \check{\nu} &= \check{\Gamma} \check{\theta}\end{aligned}\tag{1.1}$$

Introduciamo ora  $\hat{\tau}_h = \hat{\tau}_h(\hat{\eta}, \check{\eta})$  e  $\check{\tau}_h = \check{\tau}_h(\hat{\eta}, \check{\eta})$ , rispettivamente i costi della strada  $r_h$ ,  $h \in \{1, \dots, N\}$ , per la popolazione *hat* e per la popolazione *check*. I costi possono rappresentare, per esempio, il tempo necessario a percorrere la strada, il consumo di benzina oppure l'emissione di inquinanti. In questa trattazione, similmente a [3], considereremo i costi come tempi di viaggio, in quanto ciò permette di aggiungere  $+\infty$  ai valori che le funzioni di costo possono assumere, rappresentante una strada completamente congestionata o, comunque, bloccata/non percorribile:

$$\hat{\tau}, \check{\tau} : [0, 1]^N \times [0, 1]^N \rightarrow \mathbb{R}_+^N \cup \{+\infty\}$$

Per completezza, possiamo definire  $\hat{\zeta}$  e  $\check{\zeta}$ , numero totale di entità, rispettivamente della popolazione *hat* e *check*, presenti sulla rete. Allora possiamo estendere il dominio

delle funzioni:

$$\hat{\tau}, \check{\tau} : \mathbb{R}_+^N \times \mathbb{R}_+^N \rightarrow \mathbb{R}_+^N \cup \{+\infty\}$$

?

dimodoché si possano ottenere i costi delle strade, applicando:

$$\begin{aligned} \hat{\tau}(\hat{\zeta}\hat{\theta}, \check{\zeta}\check{\theta}) \\ \check{\tau}(\hat{\zeta}\hat{\theta}, \check{\zeta}\check{\theta}) \end{aligned} \quad (1.2)$$

Con un lieve abuso di notazione, indicheremo con  $\hat{\tau}$  e  $\check{\tau}$  anche l'estensione delle funzioni di costo a  $\mathbb{R}_+^N \times \mathbb{R}_+^N$ , in modo da poterle applicare direttamente a  $\hat{\zeta}\hat{\theta}$  e  $\check{\zeta}\check{\theta}$ .

Denotiamo con  $\hat{T}_i(\theta)$  e  $\check{T}_j(\theta)$  i tempi di viaggio dei percorsi  $\hat{\gamma}_i$  e  $\check{\gamma}_j$ .  $\hat{T}_i(\theta)$  e  $\check{T}_j(\theta)$  sono componenti dei vettori  $\hat{T}(\theta)$  e  $\check{T}(\theta)$ . Sfruttando anche Equazione 1.1:

$$\begin{aligned} \hat{T}(\theta) &:= \hat{\Gamma}^\top \hat{\tau}(\hat{\nu}, \check{\nu}) \quad \text{e} \quad \hat{T}_i(\theta) = \sum_{h=1}^N \hat{\Gamma}_{hi} \hat{\tau}_h(\hat{\nu}_h, \check{\nu}_h), \quad i \in \{1, \dots, \hat{n}\}; \\ \check{T}(\theta) &:= \check{\Gamma}^\top \check{\tau}(\hat{\nu}, \check{\nu}) \quad \text{e} \quad \check{T}_j(\theta) = \sum_{h=1}^N \check{\Gamma}_{hj} \check{\tau}_h(\hat{\nu}_h, \check{\nu}_h), \quad j \in \{1, \dots, \check{n}\}. \end{aligned} \quad (1.3)$$

Sarà utile, da ultimo, saper calcolare il tempo medio necessario ad attraversare la rete per la popolazione *hat* e per quella *check*:

$$\begin{aligned} \hat{T}_M(\theta) &= \hat{\theta}^\top \hat{T}(\theta) \text{ o, equivalentemente } \hat{T}_M(\theta) := \sum_{i=1}^{\hat{n}} \hat{\theta}_i \hat{T}_i(\theta); \\ \check{T}_M(\theta) &= \check{\theta}^\top \check{T}(\theta) \text{ o, equivalentemente } \check{T}_M(\theta) := \sum_{i=1}^{\check{n}} \check{\theta}_i \check{T}_i(\theta). \end{aligned} \quad (1.4)$$

Notiamo che l'Equazione 1.1 e l'Equazione 1.1 non cambiano anche nel caso in cui si utilizzino le funzioni di costo come nell'Equazione 1.2.

$\frac{1}{T}$  e  $\frac{1}{T}$  independenti

## 1.2 Equilibrio di Nash

Si riportano alcune definizioni utili da [3, Cap. 2]

**Definizione 1.1.** Dato uno stato  $\theta \in S^{\hat{n}} \times S^{\check{n}}$  chiamiamo *hat*-rilevante, rispettivamente *check*-rilevante, un tempo di viaggio  $\hat{T}_i(\theta)$ , rispettivamente  $\check{T}_i(\theta)$ , tale che  $\hat{\theta}_i \neq 0$ , rispettivamente  $\check{\theta}_i \neq 0$ .

**Definizione 1.2.** Uno stato  $\theta^* \in S^{\hat{n}} \times S^{\check{n}}$  è uno *stato di equilibrio* se tutti i tempi di viaggio rilevanti, come da 1.1, di ogni popolazione coincidono, ovvero se:

$$\begin{aligned} \forall i, j \in \{1, \dots, \hat{n}\}, \text{ se } \hat{\theta}_i^* \neq 0 \text{ e } \hat{\theta}_j^* \neq 0, \text{ allora } \hat{T}_i(\theta^*) &= \hat{T}_j(\theta^*); \\ \forall i, j \in \{1, \dots, \check{n}\}, \text{ se } \check{\theta}_i^* \neq 0 \text{ e } \check{\theta}_j^* \neq 0, \text{ allora } \check{T}_i(\theta^*) &= \check{T}_j(\theta^*). \end{aligned}$$

La 1.2 assicura che all'equilibrio tutti i guidatori di una stessa popolazione impiegano lo stesso tempo per arrivare dall'origine alla destinazione appartenenti a suddetta popolazione.

Grazie a queste definizioni ed a (Equazione 1.4), introduciamo:

**Definizione 1.3.** Uno stato di equilibrio  $\theta^*$  è un *Equilibrio di Nash* se:

$$\begin{aligned} \forall i \in \{1, \dots, \hat{n}\} \quad \hat{\theta}_i^* = 0 &\implies \hat{T}_i(\theta^*) \geq \hat{T}_M(\theta^*); \\ \forall i \in \{1, \dots, \check{n}\} \quad \check{\theta}_i^* = 0 &\implies \check{T}_i(\theta^*) \geq \check{T}_M(\theta^*). \end{aligned}$$

Notiamo che, in uno stato di equilibrio  $\theta^*$ ,  $\hat{T}_M(\theta^*) = \hat{T}_i(\theta^*) \forall i \in \{1, \dots, \hat{n}\}$  (analogo per la popolazione *check*), dato che, per la 1.2, tutti i tempi di attraversamento dei percorsi sono uguali. Perciò la 1.3 implica che un equilibrio di Nash è uno stato di equilibrio in cui a nessun guidatore di nessuna popolazione conviene cambiare percorso. Ciò è dato

?

dal fatto che in suddetto equilibrio il tempo di attraversamento dei percorsi non rilevanti è maggiore del tempo medio di attraversamento della rete, di conseguenza:

$$\begin{aligned}\forall i, j \in \{1, \dots, \hat{n}\} \quad \hat{\theta}_i^* = 0, \quad \hat{\theta}_j^* \neq 0 &\implies \hat{T}_i(\theta^*) \geq \hat{T}_j(\theta^*); \\ \forall i, j \in \{1, \dots, \check{n}\} \quad \check{\theta}_i^* = 0, \quad \check{\theta}_j^* \neq 0 &\implies \check{T}_i(\theta^*) \geq \check{T}_j(\theta^*).\end{aligned}$$

Non è detto, però, che esista un equilibrio di Nash in ogni rete. Si riporta, dunque, da [3, Cap. 2]:

**Teorema 1.4.** Supponiamo che  $\forall i \in \{1, \dots, N\} \quad (\hat{\tau}_i, \check{\tau}_i) \in C^0([0, 1]^2; (\mathbb{R}_+ \cup \{+\infty\})^2)$ . Allora esiste un equilibrio di Nash  $\theta^* \in S^{\hat{n}} \times S^{\check{n}}$ , al senso della 1.3.

Introduciamo le funzioni:

$$\begin{aligned}\bar{\varphi} : \quad \mathbb{R}_+ \cup \{+\infty\} &\rightarrow [0, 1] \\ \xi &\mapsto \begin{cases} \xi/(1-\xi) & \xi \in \mathbb{R}_+ \\ 1 & \xi = +\infty \end{cases} \\ \varphi : \quad (\mathbb{R}_+ \cup \{+\infty\})^n &\rightarrow [0, 1]^n \\ (\xi_1, \dots, \xi_n) &\mapsto (\bar{\varphi}(\xi_1), \dots, \bar{\varphi}(\xi_n)).\end{aligned} \tag{1.5}$$

Chiamiamo  $C_+^n$  l'insieme dei  $\theta \in \mathbb{R}$  tale che  $\theta_i > 0$  per almeno un  $i \in \{1, \dots, n\}$ . Definiamo quindi la normalizzazione  $\mathcal{N} : C_+^n \rightarrow S^n$ , le cui componenti sono:

$$\mathcal{N}(\theta)_i = \frac{\max\{0, \theta_i\}}{\sum_{j=1}^n \max\{0, \theta_j\}}, \text{ per } i \in \{1, \dots, n\} \tag{1.6}$$

Useremo  $\varphi$  e  $\bar{\varphi}$  indipendentemente che siano definite su  $\mathbb{R}^{\hat{n}} \cup \{+\infty\}$  o su  $\mathbb{R}^{\check{n}} \cup \{+\infty\}$ . Vale lo stesso per  $\mathcal{N}$ .



Nella dimostrazione del 1.4 presente in [3, Cap. 4] si fa utilizzo della funzione:

$$\mathcal{F} : S^{\hat{n}} \times S^{\check{n}} \rightarrow S^{\hat{n}} \times S^{\check{n}}, \quad (1.7)$$

che ha componenti:

$$\begin{aligned} \hat{\mathcal{F}} : S^{\hat{n}} \times S^{\check{n}} &\rightarrow S^{\hat{n}} \\ \theta &\mapsto \mathcal{N}(\hat{\theta} - \lambda((\varphi \circ \hat{T})(\theta) - \hat{\theta}^\top(\varphi \circ \hat{T})(\theta) \mathbf{1}_{\hat{n}})) \\ \check{\mathcal{F}} : S^{\hat{n}} \times S^{\check{n}} &\rightarrow S^{\check{n}} \\ \theta &\mapsto \mathcal{N}(\check{\theta} - \lambda((\varphi \circ \check{T})(\theta) - \check{\theta}^\top(\varphi \circ \check{T})(\theta) \mathbf{1}_{\check{n}})) \end{aligned} \quad (1.8)$$

con:

$$\lambda = \frac{1}{2} \min \left\{ \frac{1}{\hat{n}}, \frac{1}{\check{n}} \right\}. \quad (1.9)$$

Per la dimostrazione del 1.4 ci si serve del Teorema del Punto Fisso di Brouwer, usato anche da John Nash nella sua tesi di dottorato [8, Cap. 3] per dimostrare che *"Every finite game has an equilibrium point"*, punto di equilibrio denominato in seguito Equilibrio di Nash.

In [3, Cap. 4] viene dunque verificato che (Equazione 1.7), per Brouwer, ammette almeno un punto fisso:

$$\exists \theta^* \in S^{\hat{n}} \times S^{\check{n}} : \mathcal{F}(\theta^*) = \theta^* \quad (1.10)$$

e che ogni punto fisso di  $\mathcal{F}$  è un Equilibrio di Nash per la rete stradale, al senso della 1.3.

Notiamo che l'analisi effettuata in questa sezione non cambia se si utilizzano le funzioni di costo come nell'Equazione 1.2.

## 1.3 Esempi

Il programma riportato in sezione 2, al seguito della creazione di una classe Python [10] rappresentante la rete stradale studiata, può essere utilizzato per 3 diversi scopi:

- Calcolo dell'equilibrio di Nash e del tempo di attraversamento della rete
- Computazione dei grafici del tempo di attraversamento della rete e dell'equilibrio di Nash dipendenti dalla variazione dei costi delle strade
- Computazione del grafico del tempo di attraversamento della rete e dell'equilibrio di Nash dipendenti dalla variazione delle popolazioni

*PRINT* Come "tempo di attraversamento della rete" si intende, distintamente per le due popolazioni, il valore assunto dai  $T$  rilevanti (1.1) che caratterizzano l'Equilibrio di Nash trovato. *Common*

### 1.3.1 Inizializzazione Programma

Per utilizzare il programma (vedi sezione 2) scriviamo il file **main.py**:

Listato 1: main.py

```
1 from pathlib import Path
2 import importlib
3 from Initializer import Initializer
4
5 def main():
6     Param = importlib.import_module(f"{getParam()}.Parameters")
7     param = Param.Parameters()
8
9     #Vedi Listato 9
10    init = Initializer(param)
11    init.start()
```

```

12
13 def getParam():
14     cartella = Path(__file__).parent.joinpath("parameters")
15     parametri = [f.name for f in cartella.iterdir() if f.is_dir() and f.name.startswith("
        parameters")]
16     print(parametri)
17
18     scelta = None
19     while scelta is None:
20         try:
21             indice = int(input("Scegli un pacchetto (numero): ")) - 1
22             if 0 <= indice < len(parametri):
23                 scelta = parametri[indice]
24             else:
25                 print("Numero non valido. Riprova.")
26         except ValueError:
27             print("Inserisci un numero valido.")
28     return "parameters." + scelta
29
30 if __name__ == "__main__":
31     main()

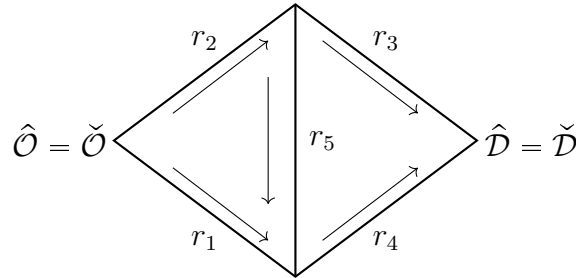
```

Grazie alla funzione **getParam**, possiamo organizzare i diversi parametri dei casi di studio salvandoli nella cartella **parameters** situata nella stessa directory del file **main.py**, ovvero come **./parameters/parameters*i*/Parameters.py**, dove *i* è il numero progressivo utilizzato per selezionare quale **Parameters.py** utilizzare una volta avviato il programma. **Parameters.py** utilizzare.

Spiegare!

### 1.3.2 Esempio 1

Introduciamo, come descritta in [3, Cap. 3], la rete stradale così formata:



Analizziamo l'effetto della variazione del costo della strada  $r_5$ ; passiamo da  $\hat{\tau}_5(\eta) = \check{\tau}_5(\eta) = 0$  a  $\hat{\tau}_5(\eta) = \check{\tau}_5(\eta) = 50$ . Istanziamo dunque un nuovo **Parameters.py** (vedi Listato 6 per specifica delle **@property**):

Listato 2: Parameters.py

```
1 from typing import override
2
3 from utility.SafeArray import SafeArray #Vedi Listato 8
4 from utility.Operations import Operations #Vedi Listato 7
5 from utility.AbstractParam import AbstractParam
6
7 class Parameters(AbstractParam):
8
9     @property
10     def operation(self):
11         return Operations.NASH_EQ_TIME_VARIATIONS
12
13     @property
14     def output_directory(self):
15         #Directory di salvataggio dei dati uguale a quella di Parameters.py
16         return "parameters\\parameters3"
17
18     @property
```

```

19     def MIN(self) -> float:
20         return 0.
21
22     @property
23     def MAX(self) -> float:
24         return 50.
25
26     @property
27     def step(self) -> float:
28         return 0.5
29
30     @property
31     def variation_hat(self) -> SafeArray:
32         return SafeArray([0., 0., 0., 0., 1.])
33
34     @property
35     def variation_check(self) -> SafeArray:
36         return SafeArray([0., 0., 0., 0., 1.])
37
38     @property #  $\hat{\Gamma}$ 
39     def Gamma_hat(self):
40         return SafeArray([
41             [0., 1., 0.],
42             [1., 0., 1.],
43             [1., 0., 0.],
44             [0., 1., 1.],
45             [0., 0., 1.]
46         ])
47
48     @property #  $\check{\Gamma}$ 
49     def Gamma_check(self):
50         return SafeArray([
51             [0., 1., 0.],
52             [1., 0., 1.],
53             [1., 0., 0.],
54             [0., 1., 1.],
55             [0., 0., 1.]

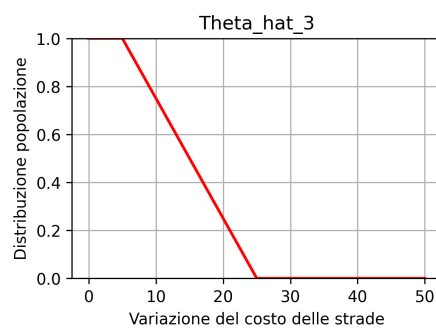
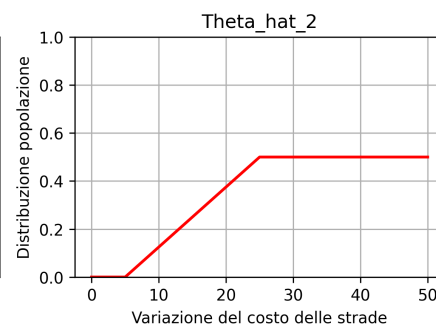
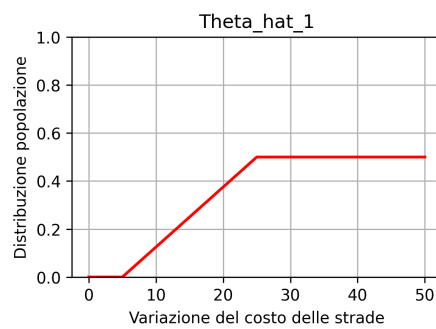
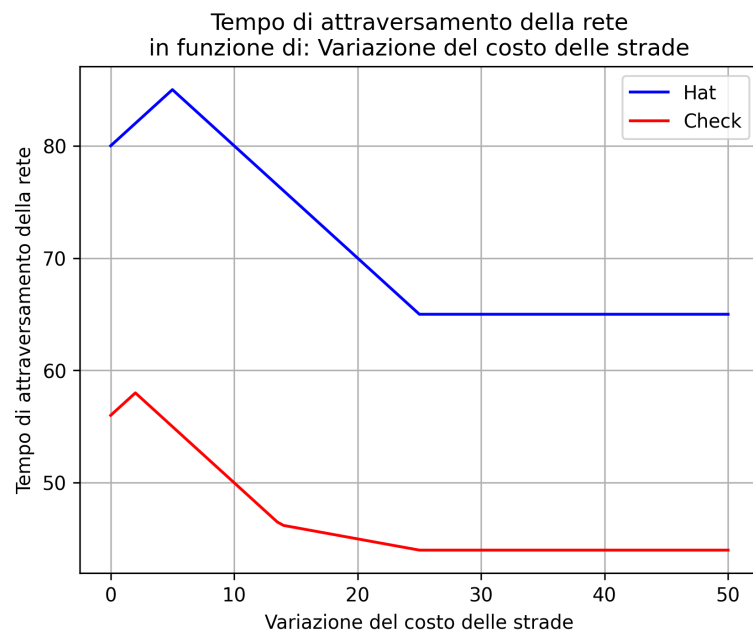
```

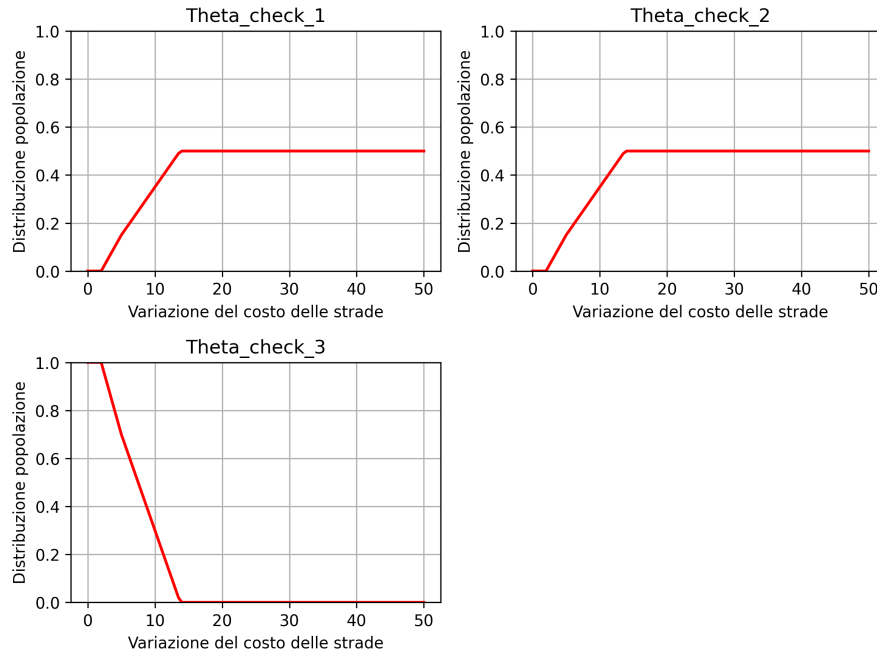
```

56         ])
57
58     @override # $\hat{\tau}(\hat{\eta}, \check{\eta})$ 
59     def tau_hat(self, eta_hat, eta_check):
60         return (
61             SafeArray([
62                 45.,
63                 40. * eta_hat[1, 0],
64                 45.,
65                 40. * eta_hat[3, 0],
66                 0
67             ])
68         )
69
70     @override # $\check{\tau}(\hat{\eta}, \check{\eta})$ 
71     def tau_check(self, eta_hat, eta_check):
72         return (
73             SafeArray([
74                 30.,
75                 20. * eta_check[1, 0] + 8. * eta_hat[1, 0],
76                 30.,
77                 20. * eta_check[1, 0] + 8. * eta_hat[1, 0],
78                 0
79             ])
80         )

```

Eseguendo il programma troviamo i seguenti grafici:





Con l'aumentare del costo della strada  $r_5$  si ottiene un risultato inatteso: il tempo di attraversamento della rete diminuisce. Progressivamente sempre più entità delle due popolazioni scelgono i percorsi che non contengono la strada  $r_5$ , finché nessuno la utilizza più, raggiungendo il minimo del tempo di attraversamento.

La rimozione di suddetta strada migliora la qualità del viaggio sulla rete: ci troviamo davanti a un caso di Paradosso di Braess [1]. Questa situazione è molto fragile: se esaminiamo il caso in cui  $\hat{\tau}_5(\eta) = \check{\tau}_5(\eta) = 0$ , facendo variare il numero di entità delle popolazioni, da  $\hat{\zeta} = \check{\zeta} = 1$  a  $\hat{\zeta} = \check{\zeta} = 3$ :

Listato 3: Parameters.py

```

1 from typing import override
2
3 from utility.SafeArray import SafeArray
4 from utility.Operations import Operations

```



```

5 from utility.AbstractParam import AbstractParam
6
7 class Parameters(AbstractParam):
8
9     @property
10    def operation(self):
11        return Operations.NASH_EQ_POP_VARIATIONS
12
13    @property
14    def output_directory(self):
15        return "parameters\\parameters3"
16
17    @property
18    def MIN(self) -> float:
19        """
20        La variazione parte da self.entity_number_hat (o check), definito come 1 se non
21        sovrascritto.
22        Vedi Listato 6
23        """
24        return 0.
25
26    @property
27    def MAX(self):
28        """
29        La variazione arriva aa self.entity_number_hat (o check), che vale 3 in questo caso
30        Vedi Listato 6
31        """
32        return 2.
33
34    @property
35    def step(self) -> float:
36        return 0.01
37
38    @property
39    def Gamma_hat(self):
40        return SafeArray([
            [0., 1., 0.],

```

```

41         [1., 0., 1.],
42         [1., 0., 0.],
43         [0., 1., 1.],
44         [0., 0., 1.]
45     ])
46
47     @property
48     def Gamma_check(self):
49         return SafeArray([
50             [0., 1., 0.],
51             [1., 0., 1.],
52             [1., 0., 0.],
53             [0., 1., 1.],
54             [0., 0., 1.]
55         ])
56
57     @override
58     def tau_hat(self, eta_hat, eta_check):
59         return (
60             SafeArray([
61                 45.,
62                 40. * eta_hat[1, 0],
63                 45.,
64                 40. * eta_hat[3, 0],
65                 0
66             ])
67         )
68
69     @override
70     def tau_check(self, eta_hat, eta_check):
71         return (
72             SafeArray([
73                 30.,
74                 20. * eta_check[1, 0] + 8. * eta_hat[1, 0],
75                 30.,
76                 20. * eta_check[1, 0] + 8. * eta_hat[1, 0],
77                 0

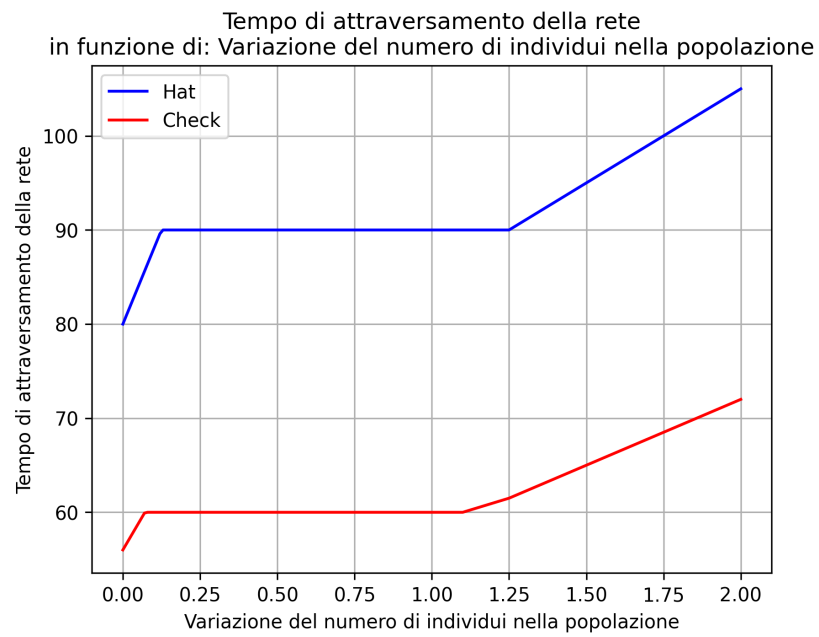
```

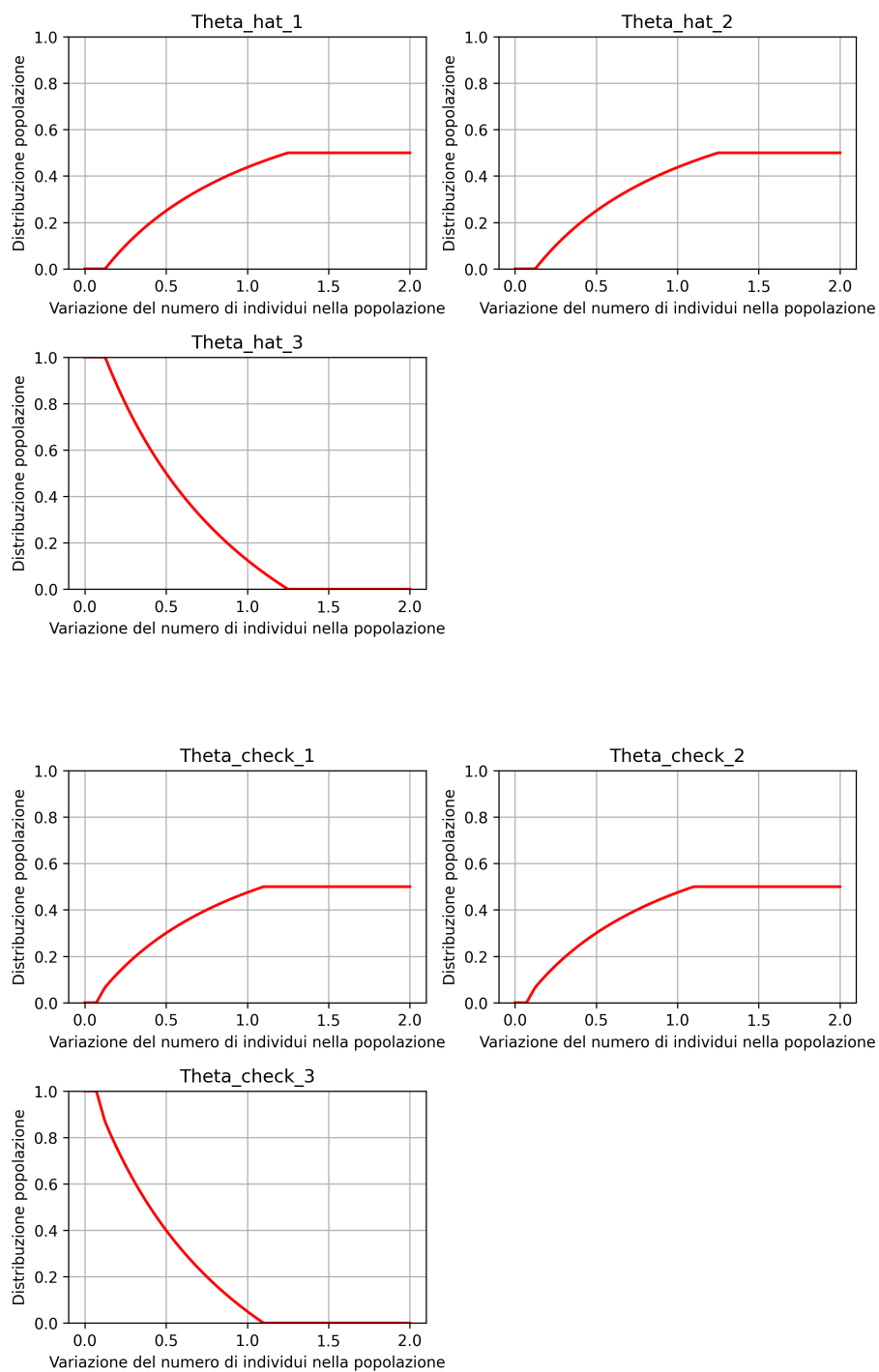
78  
79

1)

)

otteniamo:



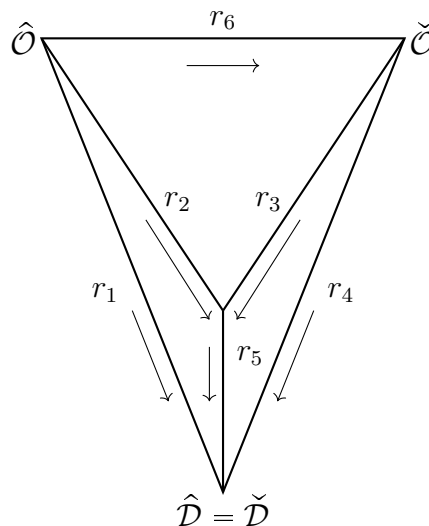


Aumentando le entità delle due popolazioni si ottiene, a seguito di una breve crescita del tempo di attraversamento, una fase transitoria costante, in cui le due popolazioni iniziano progressivamente ad utilizzare i due percorsi che non contengono  $r_5$ . Terminato il transitorio costante nessuno utilizza  $r_5$  e si perde il comportamento paradossale della rete: anche se la strada ha costo 0 non conviene attraversarla.

?

### 1.3.3 Esempio 2

Introduciamo, sempre riprendendola da [3, Cap. 3], la seguente rete:



Variando il costo della strada  $r_6$  solamente per la popolazione *hat*, da  $\hat{\tau}_6(\eta) = 1$  a  $\hat{\tau}_6(\eta) = 1.5$ :

Listato 4: Parameters.py

```
1 from typing import override
2
3 from utility.SafeArray import SafeArray
4 import numpy as np
```

```

5 from utility.Operations import Operations
6 from utility.AbstractParam import AbstractParam
7
8 class Parameters(AbstractParam):
9
10     @property
11     def operation(self):
12         return Operations.NASH_EQ_TIME_VARIATIONS
13
14     @property
15     def output_directory(self) -> str:
16         return "parameters\\parameters4"
17
18     @property
19     def MIN(self):
20         return 0.
21
22     @property
23     def MAX(self):
24         return 0.5
25
26     @property
27     def step(self):
28         return 0.01
29
30     @property
31     def Gamma_hat(self):
32         return SafeArray([[1., 0., 0.],
33                             [0., 1., 0.],
34                             [0., 0., 0.],
35                             [0., 0., 1.],
36                             [0., 1., 0.],
37                             [0., 0., 1.]])
38
39     @property
40     def Gamma_check(self) -> SafeArray:
41         return SafeArray([[0., 0.],

```

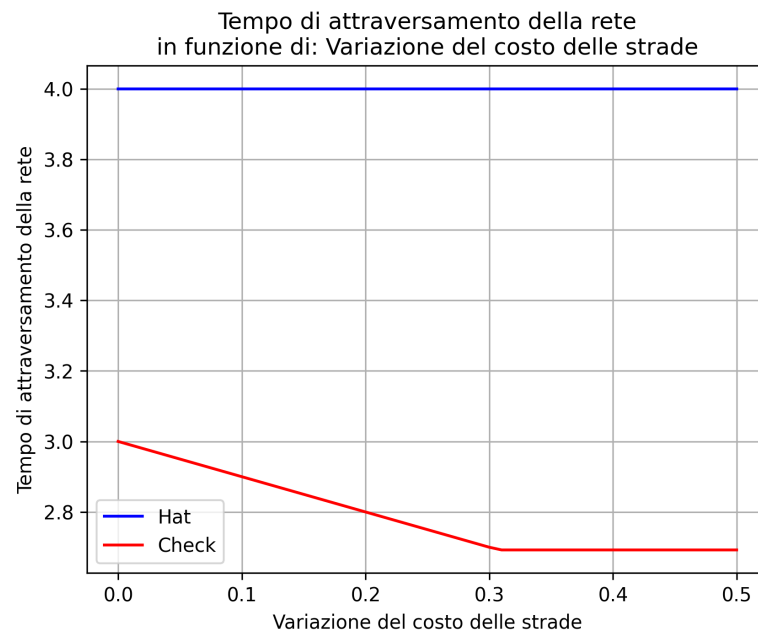
```

42         [0., 0.],
43         [1., 0.],
44         [0., 1.],
45         [1., 0.],
46         [0., 0.]]
47
48     @property
49     def variation_hat(self):
50         return SafeArray([0., 0., 0., 0., 0., 1.])
51
52     @property
53     def variation_check(self):
54         return SafeArray([0., 0., 0., 0., 0., 0.])
55
56     @override
57     def tau_hat(self, eta_hat, eta_check):
58         return (
59             SafeArray([
60                 4.,
61                 1.+eta_hat[1, 0],
62                 np.inf,
63                 5.*eta_hat[3, 0]+5.*eta_check[3, 0],
64                 1.+eta_hat[4, 0]+eta_check[4, 0],
65                 1
66             ])
67         )
68
69     @override
70     def tau_check(self, eta_hat, eta_check):
71         return (
72             SafeArray([
73                 np.inf,
74                 np.inf,
75                 eta_check[2, 0],
76                 5.*eta_hat[3, 0]+5.*eta_check[3, 0],
77                 1.+eta_hat[4, 0]+eta_check[4, 0],
78                 np.inf

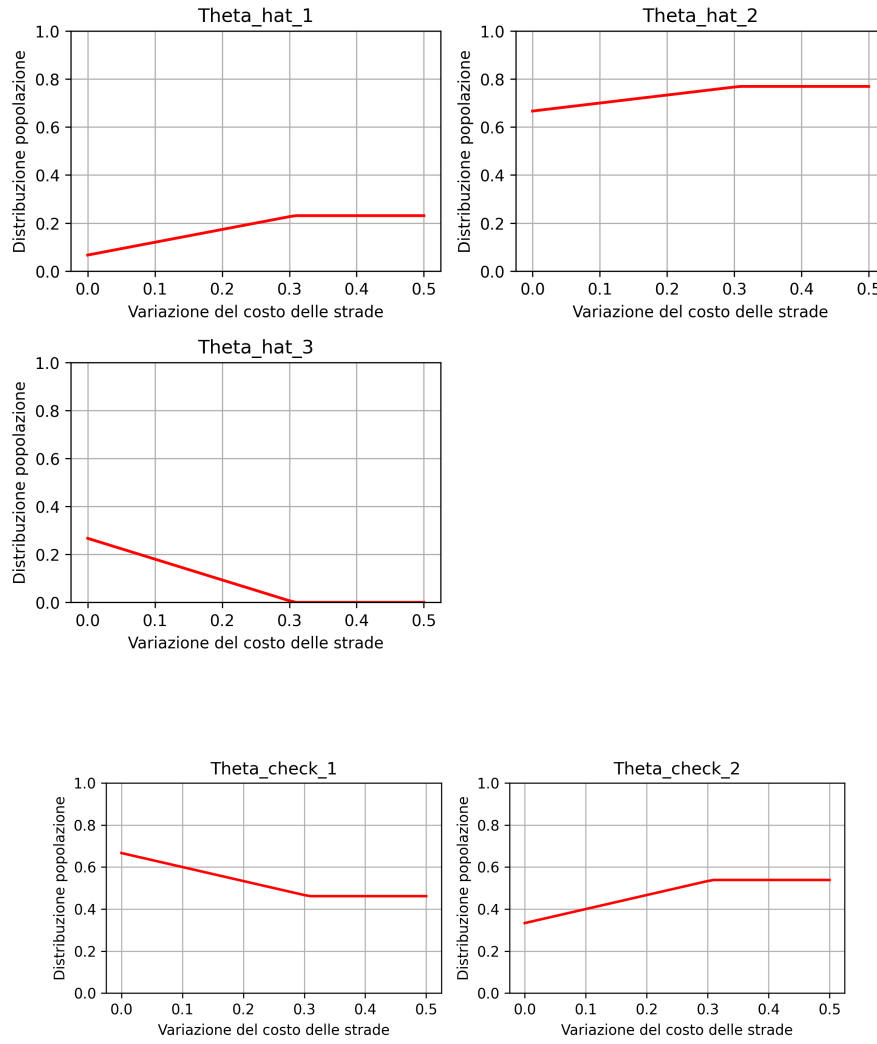
```

|    |    |
|----|----|
| 79 | 1) |
| 80 | )  |

otteniamo:







Come nella sottosezione 1.3.2 osserviamo un Paradosso di Braess [1]. Questa volta, però, solo una delle due popolazioni (*check*) risente della variazione di costo di  $r_6$ , nonostante nessuna entità *check* la attraversi: l'aggiunta di  $r_6$ , che non migliora il tempo di attraversamento dell'unica popolazione che ne usufruisce (*hat*), peggiora invece il tempo di attraversamento dell'altra.

Nuovamente si nota che questo stato paradossale è fragile: facendo variare, per esem-

pio, il numero di entità nella popolazione *check*, da  $\check{\zeta} = 1$  a  $\check{\zeta} = 6$ :

Listato 5: Parameters.py

```
1 from typing import override
2
3 from utility.SafeArray import SafeArray
4 import numpy as np
5 from utility.Operations import Operations
6 from utility.AbstractParam import AbstractParam
7
8 class Parameters(AbstractParam):
9
10     @property
11     def operation(self):
12         return Operations.NASH_EQ_POP_VARIATIONS
13
14     @property
15     def output_directory(self) -> str:
16         return "parameters\\parameters4"
17
18     @property
19     def MIN(self):
20         """
21         La variazione parte da self.entity_number_check, definito come 1 se non sovrascritto.
22         Vedi Listato 6
23         """
24         return 0.
25
26     @property
27     def MAX(self):
28         """
29         La variazione arriva aa self.entity_number_check, che vale 6 in questo caso
30         Vedi Listato 6
31         """
32         return 5.
33
```

```

34     @property
35     def step(self):
36         return 0.05
37
38     @property
39     def variate_pop_hat(self) -> bool:
40         return False
41
42     @property
43     def variate_pop_check(self) -> bool:
44         return True
45
46     @property
47     def Gamma_hat(self):
48         return SafeArray([[1., 0., 0.],
49                             [0., 1., 0.],
50                             [0., 0., 0.],
51                             [0., 0., 1.],
52                             [0., 1., 0.],
53                             [0., 0., 1.]])
54
55     @property
56     def Gamma_check(self) -> SafeArray:
57         return SafeArray([[0., 0.],
58                             [0., 0.],
59                             [1., 0.],
60                             [0., 1.],
61                             [1., 0.],
62                             [0., 0.]])
63
64     @override
65     def tau_hat(self, eta_hat, eta_check):
66         return (
67             SafeArray([
68                 4.,
69                 1.+eta_hat[1, 0],
70                 np.inf,

```

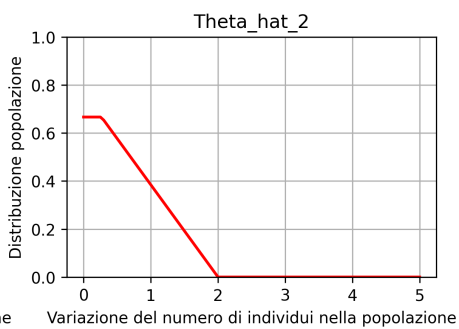
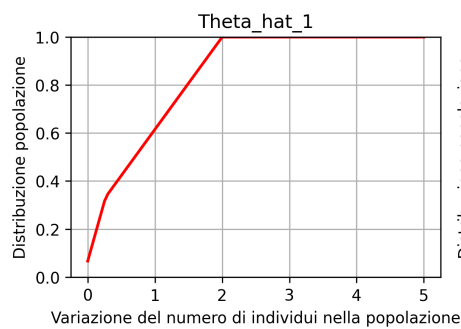
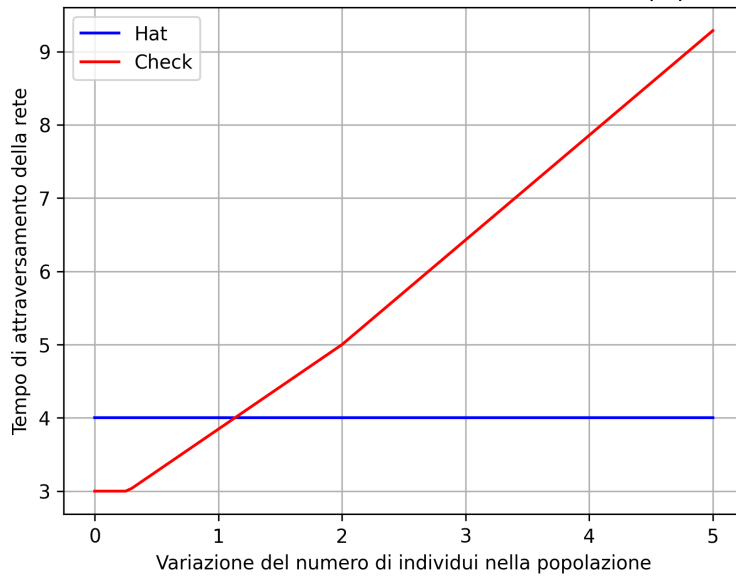
```

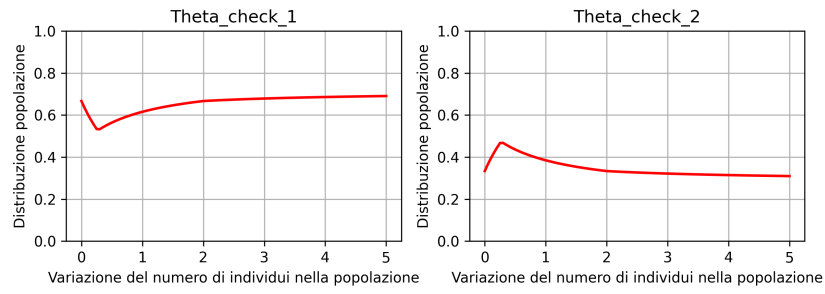
71         5.*eta_hat[3, 0]+5.*eta_check[3, 0],
72         1.+eta_hat[4, 0]+eta_check[4, 0],
73         1
74     ])
75 )
76
77 @override
78 def tau_check(self, eta_hat, eta_check):
79     return (
80         SafeArray([
81             np.inf,
82             np.inf,
83             eta_check[2, 0],
84             5.*eta_hat[3, 0]+5.*eta_check[3, 0],
85             1.+eta_hat[4, 0]+eta_check[4, 0],
86             np.inf
87         ])
88     )

```

risulta:

Tempo di attraversamento della rete  
in funzione di: Variazione del numero di individui nella popolazione





Durante un transitorio iniziale del tempo di percorrenza la popolazione *hat* smette progressivamente di utilizzare la  $r_6$  e, nonostante la popolazione *check* aumenti, il suo tempo di percorrenza rimane costante: viene perso il comportamento paradossale della rete.

## 2 Programma

Di seguito viene riportato il codice sorgente del programma utilizzato per la trattazione.

### 2.1 Contratto AbstractParam

Per il corretto utilizzo del programma e' necessario creare una classe che estende **AbstractParam**:

Listato 6: AbstractParam

```
1 class AbstractParam(ABC):
2     """
3     Contratto obbligatorio per i Parametri
4     """
5
6     # ===== PARAMETRI BASE =====
7
8     @property
9     @final
10    def n_roads(self) -> int:          #N
11        """
12        Numero totale di strade del grafo.
13
14        Returns:
15            int: numero di righe di Gamma_hat (una per ogni strada)
16        """
17        return self.Gamma_hat.shape[0]
18
19    @property
20    @final
21    def n_routes_hat(self) -> int:     #n̂
22        """
23        Numero di percorsi disponibili per la popolazione hat.
24
25        Returns:
```

```

26         int: numero di colonne di Gamma_hat
27         """
28         return self.Gamma_hat.shape[1]
29
30     @property
31     @final
32     def n_routes_check(self) -> int:      #ñ
33         """
34         Numero di percorsi disponibili per la popolazione check.
35
36         Returns:
37             int: numero di colonne di Gamma_check
38         """
39         return self.Gamma_check.shape[1]
40
41     @property
42     @abstractmethod
43     def operation(self) -> Operations:
44         """
45         Tipo di operazione da eseguire (es. Nash equilibrium).
46
47         Returns:
48             Operations: tipo di operazione
49         """
50         pass
51
52     @property
53     @abstractmethod
54     def output_directory(self) -> str:
55         """
56         Directory dove salvare i risultati.
57
58         Returns:
59             str: percorso della directory di output
60         """
61         pass
62

```



```

63     @property
64     def save_result(self) -> bool:
65         """
66         Indica se salvare i risultati su file.
67
68         Returns:
69             bool: True se i risultati devono essere salvati
70         """
71         return False
72
73     @property
74     def show_iterations(self) -> bool:
75         """
76         Indica se mostrare le iterazioni durante il calcolo.
77
78         Returns:
79             bool: True se si vogliono stampare a terminale le iterazioni
80         """
81         return True
82
83     @property
84     def save_as_fraction(self) -> bool:
85         """
86         Indica se salvare i valori come frazioni.
87         Utilizzato solo nel caso in cui self.operation==Operations.NASH_EQ
88
89         Returns:
90             bool: True se i numeri devono essere convertiti in frazioni
91         """
92         return True
93
94     @property
95     def MIN(self) -> float:
96         """
97         Valore minimo della variazione nei grafici.
98
99         Returns:

```

```

100         float: limite inferiore
101     """
102     return 0.
103
104 @property
105 def MAX(self) -> float:
106     """
107     Valore massimo della variazione nei grafici.
108
109     Returns:
110         float: limite superiore
111     """
112     return 1.
113
114 @property
115 def step(self) -> float:
116     """
117     Passo della variazione nei grafici.
118
119     Returns:
120         float: incremento della variazione
121     """
122     return 0.01
123
124 @property
125 def entity_number_hat(self) -> float:    # $\hat{\zeta}$ 
126     """
127     Numero totale di entità nella popolazione hat.
128     Nel caso in cui self.operation==Operations.NASH_EQ_POP_VARIATIONS, viene
129     utilizzato solo se self.variate_pop_hat==False
130
131     Returns:
132         float: numero di entità hat
133     """
134     return 1.
135
136 @property

```

```

137 def entity_number_check(self) -> float: #ζ
138     """
139     Numero totale di entità nella popolazione check.
140     Nel caso in cui self.operation==Operations.NASH_EQ_POP_VARIATIONS, viene
141     utilizzato solo se self.variate_pop_check==False
142
143     Returns:
144         float: numero di entità check
145     """
146     return 1.
147
148 @property
149 def variate_pop_hat(self) -> bool:
150     """
151     Indica se variare il numero di entità della popolazione hat durante il calcolo,
152     nel caso in cui self.operation==Operations.NASH_EQ_POP_VARIATIONS
153
154     Returns:
155         bool: True se si deve variare il numero di entità della popolazione hat
156     """
157     return True
158
159 @property
160 def variate_pop_check(self) -> bool:
161     """
162     Indica se variare il numero di entità della popolazione check durante il calcolo,
163     nel caso in cui self.operation==Operations.NASH_EQ_POP_VARIATIONS
164
165     Returns:
166         bool: True se si deve variare il numero di entità della popolazione check
167     """
168     return True
169
170
171
172 # ===== MATRICI =====
173

```

```

174 @property
175 @abstractmethod
176 def Gamma_hat(self) -> SafeArray: # $\hat{\Gamma}$ 
177     """
178     Matrice dei percorsi per la popolazione hat.
179
180     Returns:
181         SafeArray: matrice (n_roads x n_routes_hat)
182     """
183     pass
184
185 @property
186 @abstractmethod
187 def Gamma_check(self) -> SafeArray: # $\check{\Gamma}$ 
188     """
189     Matrice dei percorsi per la popolazione check.
190
191     Returns:
192         SafeArray: matrice (n_roads x n_routes_check)
193     """
194     pass
195
196 @property
197 @abstractmethod
198 def variation_hat(self) -> SafeArray:
199     """
200     Ritorna array di dimensione  $N$ , i cui elementi sono 1 o 0:
201     - 'return[i]!=0': se viene utilizzata una variazione, i costi della strada 'i'
202       variano per la popolazione check di un coefficiente 'return[i]', maggiore
203       o minore di 0
204     - 'return[i']==0': se viene utilizzata una variazione, i costi della strada 'i'
205       non variano per la popolazione hat
206     """
207     pass
208
209 @property
210 @abstractmethod

```

```

208 def variation_check(self) -> SafeArray:
209     """
210     Ritorna array di dimensione  $N$ , i cui elementi sono:
211     - 'return[i']!=0: se viene utilizzata una variazione, i costi della strada 'i'
212       variano per la popolazione check di un coefficiente 'return[i]', maggiore
213       o minore di 0
214     - 'return[i']==0: se viene utilizzata una variazione, i costi della strada 'i'
215       non variano per la popolazione check
216     """
217
218     pass
219
220     # ===== COSTI =====
221
222     @abstractmethod
223     def tau_hat(self, eta_hat, eta_check) -> SafeArray:       $\hat{\tau}(\hat{\eta}, \check{\eta})$ 
224     """
225     Calcola il costo (tempo di viaggio) per la popolazione hat.
226
227     Args:
228         eta_hat: entità hat presenti su ogni strada
229         eta_check: entità check presenti su ogni strada
230
231     Returns:
232         SafeArray: costo per ogni strada (dimensione n_roads)
233     """
234
235     pass
236
237     @abstractmethod
238     def tau_check(self, eta_hat, eta_check) -> SafeArray:     $\check{\tau}(\hat{\eta}, \check{\eta})$ 
239     """
240     Calcola il costo (tempo di viaggio) per la popolazione check.
241
242     Args:
243         eta_hat: entità hat presenti su ogni strada
244         eta_check: entità check presenti su ogni strada

```

```

242
243     Returns:
244         SafeArray: costo per ogni strada (dimensione n_roads)
245     """
246     pass
247
248 @final
249 def tau_hat_variated(self, eta_hat, eta_check, variation) -> SafeArray:
250     """
251     Calcola il costo per la popolazione hat includendo una variazione lineare.
252
253     Il costo restituito è:
254         tau_hat + variation_hat * variation
255
256     Args:
257         eta_hat: entità hat presenti su ogni strada
258         eta_check: entità check presenti su ogni strada
259         variation: coefficiente scalare di variazione
260
261     Returns:
262         SafeArray: costo modificato per ogni strada
263     """
264     return self.tau_hat(eta_hat, eta_check) + self.variation_hat * variation
265
266 @final
267 def tau_check_variated(self, eta_hat, eta_check, variation) -> SafeArray:
268     """
269     Calcola il costo per la popolazione check includendo una variazione lineare.
270
271     Il costo restituito è:
272         tau_check + variation_check * variation
273
274     Args:
275         eta_hat: entità hat presenti su ogni strada
276         eta_check: entità check presenti su ogni strada
277         variation: coefficiente scalare di variazione
278

```

```

279         Returns:
280             SafeArray: costo modificato per ogni strada
281         """
282         return self.tau_check(eta_hat, eta_check) + self.variation_check * variation

```

Le proprietà e i metodi in Listato 6 contrassegnati da **@abstractmethod** devono essere istanziati nella classe dei parametri creata dall'utente, quelli contrassegnati da **@final** non devono essere istanziati, mentre i restanti hanno valori base che possono essere reistanziati o meno.

## 2.2 Utility

Viene introdotta una Enum per permettere all'utente di definire quale operazione eseguire (tramite il Listato 6):

- Calcolo dell'equilibrio di Nash
- Computazione dei grafici del tempo di percorrenza della rete e dell'equilibrio di Nash dipendenti dalla variazione dei costi delle strade (definita sempre nel Listato 6)
- Computazione del grafico del tempo di percorrenza della rete e dell'equilibrio di Nash dipendenti dalla variazione delle popolazioni (definita sempre nel Listato 6)

Listato 7: Operations

```

1  class Operations(Enum):
2      NASH_EQ = 1
3      NASH_EQ_TIME_VARIATIONS = 2
4      NASH_EQ_POP_VARIATIONS = 3

```

Estendiamo anche la classe **np.ndarray** (vedi [9]) per gestire i casi in cui un valore di un array/matrice posto ad **np.inf** viene moltiplicato per 0. Ciò può capitare soprattutto durante il calcolo dell'Equazione 1.3

Listato 8: SafeArray

```
1  import numpy as np
2
3  class SafeArray(np.ndarray):
4      def __new__(cls, input_array):
5          obj = np.asarray(input_array, dtype=float).view(cls)
6          return obj
7
8      def __matmul__(self, other):
9          selfChanged = False
10         otherChanged = True
11
12         if np.isinf(self).any():
13             self[np.isinf(self)] = 1.e200
14             selfChanged = True
15         if np.isinf(other).any():
16             other[np.isinf(other)] = 1.e200
17             otherChanged = False
18
19         result = np.matmul(self, other)
20
21         result[result > 1.e100] = np.inf
22
23         if selfChanged:
24             self[self>1.e100] = np.inf
25         if otherChanged:
26             other[other>1.e100] = np.inf
27
28         return result.view(type(self))
```



## 2.3 Classe Initializer

Una volta creata una classe che estende **AbstractParam**, per eseguire il programma è sufficiente creare un'istanza di **Initializer** e invocare il metodo **Initializer::start**:

Listato 9: Initializer

```
1 class Initializer:
2
3     def __init__(self, param: AbstractParam):
4         np.seterr(divide='ignore')
5
6         if not isinstance(param, AbstractParam):
7             raise TypeError("param deve essere un'istanza di BaseParameters")
8
9         self.param = param
10        self.computer = Computer(self.param) #Vedi Listato 15
11        self.initial_theta_hat = SafeArray(np.ones((self.param.n_routes_hat, 1))*(1/self.param
            .n_routes_hat))
12        self.initial_theta_check = SafeArray(np.ones((self.param.n_routes_check, 1))*(1/self.
            param.n_routes_check))
13        self.limit_hat = 1e-11
14        self.limit_check = 1e-11
15
16
17    def start(self):
18        """
19        In base a operation in param (vedi Listato 6), decide se calcolare l'equilibrio di
20        Nash (senza variazioni) o costruire il grafico
21        variando i costi delle strade
22        """
23        if self.param.operation == Operations.NASH_EQ:
24            return self.getNashEquilibria()
25        else:
26            return self.graphVariations()
```

Il metodo **Initializer::getNashEquilibria** calcola l'equilibrio di Nash utilizzando i

dati forniti tramite **param** (istanza di Listato 6), senza variare i costi delle strade e le entità nelle popolazioni. Decide inoltre se salvare il risultati su file verificando il valore attribuito a **param.save\_result**. Ritorna, in ogni caso, i valori che descrivono l'equilibrio di Nash della rete.

Listato 10: `Initializer::getNashEquilibria`

```

1 def getNashEquilibria(self):
2     """
3         Calcola l'Equilibrio di Nash e ritorna la distribuzione della popolazione hat e check
4         sulla rete, insieme ai tempi di percorrenza della stessa.
5         Se save_result==True, stampa tali risultati
6     """
7     #Equilibrio di Nash:
8     theta_hat, theta_check = self.computer.getNashEquilibria(self.initial_theta_hat, self.
9         initial_theta_check, self.limit_hat, self.limit_check)
10
11     #Tempi di viaggio su ogni strada:
12     T_hat = self.computer.T_hat(theta_hat, theta_check, self.param.entity_number_hat, self.
13         param.entity_number_check, 0)
14     T_check = self.computer.T_check(theta_hat, theta_check, self.param.entity_number_hat, self.
15         param.entity_number_check, 0)
16
17     if self.param.save_result: self.saveNashEq(theta_hat, theta_check, T_hat, T_check)
18
19     return theta_hat, theta_check, T_hat, T_check

```

Con:

Listato 11: `Initializer::saveNashEq`

```

1 def saveNashEq(self, theta_hat, theta_check, T_hat, T_check, file_name="nash_eq.txt"):
2     content = self._format_nash_eq(theta_hat, theta_check, T_hat, T_check)
3
4     file_path = Path(self.param.output_directory) / file_name
5

```

```

6      with open(file_path, "w", encoding="utf-8") as f:
7          f.write(content)
8
9  def _format_nash_eq(self, theta_hat, theta_check, T_hat, T_check):
10      vec_frac = np.vectorize(
11          lambda x: str(x) if np.isinf(x) or not self.param.save_as_fraction
12          else str(Fraction(x).limit_denominator(1000000))
13      )
14
15      lines = []
16      lines.append("Equilibrio di Nash:")
17      lines.append(f"theta_hat:\n{vec_frac(theta_hat)}")
18      lines.append(f"theta_check:\n{vec_frac(theta_check)}")
19      lines.append("")
20      lines.append("Tempi di attraversamento dei percorsi:")
21      lines.append(f"T_hat:\n{vec_frac(T_hat)}")
22      lines.append(f"T_check:\n{vec_frac(T_check)}")
23
24      return "\n".join(lines)

```

Il metodo **Initializer::graphVariations** calcola i parametri utili alla costruzione dei grafici rappresentanti la variazione del tempo di attraversamento della rete e la variazione dell'equilibrio di Nash rispetto alla variazione dei costi delle strade o del numero di entità delle popolazioni definite in **param**, Listato 6. Decide se costruire tali grafici e salvarli in **param.output\_directory** verificando il valore attribuito a **param.save\_result**. Ritorna, in ogni caso, i valori utili a disegnarli.

Listato 12: Initializer::graphVariations

```

1  def graphVariations(self):
2      """
3      Calcola gli array per costruire i grafici del tempo di percorrenza della rete e dell'
        equilibrio di Nash
4      per le due popolazioni in funzione della variazione (costi o popolazione).
5      Restituisce gli array per costruire i grafici e opzionalmente salva i risultati.

```

```

6      """
7      p = self.param
8      variation_values = np.arange(p.MIN, p.MAX + p.step, p.step)
9      N = len(variation_values)
10
11     time_hat = np.zeros(N)
12     time_check = np.zeros(N)
13     thetas_hat = np.zeros((p.n_routes_hat, N))
14     thetas_check = np.zeros((p.n_routes_check, N))
15
16     theta_hat, theta_check = self.initial_theta_hat, self.initial_theta_check
17
18     for idx, v in enumerate(variation_values):
19         if p.show_iterations:
20             print(f"Current step: {idx+1}/{N}")
21
22         #!!!!!!!!!!!!!!!!!!!!!!
23         theta_hat, theta_check, T_hat, T_check = self._compute_step(
24             theta_hat, theta_check, v
25         )
26
27         i_hat = np.argmax(theta_hat > 0)
28         i_check = np.argmax(theta_check > 0)
29
30         time_hat[idx] = T_hat[i_hat]
31         time_check[idx] = T_check[i_check]
32         thetas_hat[:, idx] = theta_hat[:, 0]
33         thetas_check[:, idx] = theta_check[:, 0]
34
35     if p.save_result:
36         label = ('Variazione del costo delle strade'
37                 if p.operation == Operations.NASH_EQ_TIME_VARIATIONS
38                 else 'Variazione del numero di individui nella popolazione')
39
40     #Grafico del tempo di percorrenza della rete in funzione dalla variazione
41     self.saveResultVariation(variation_values, time_hat, time_check, label)
42

```

```

43         #Grafici dell'equilibrio di Nash in funzione della variazione
44         self.saveResultThetaVariation(variation_values, thetas_hat, 'hat', label)
45         self.saveResultThetaVariation(variation_values, thetas_check, 'check', label)
46
47     return variation_values, time_hat, time_check, thetas_hat, thetas_check

```

Con:

### Listato 13: Initializer::\_compute\_step

```

1 def _compute_step(self, theta_hat, theta_check, v):
2     p = self.param
3     c = self.computer
4
5     if p.operation == Operations.NASH_EQ_TIME_VARIATIONS:
6         theta_hat, theta_check = c.getNashEquilibriaCostVariation( #Vedi Listato 19
7             theta_hat, theta_check, self.limit_hat, self.limit_check, v
8         )
9         pop_hat, pop_check, var = p.entity_number_hat, p.entity_number_check, v
10
11     else:
12         pop_hat = p.entity_number_hat + (v if p.variate_pop_hat else 0)
13         pop_check = p.entity_number_check + (v if p.variate_pop_check else 0)
14
15         theta_hat, theta_check = c.getNashEquilibriaPopVariation( #Vedi Listato 19
16             theta_hat, theta_check, self.limit_hat, self.limit_check, pop_hat, pop_check
17         )
18         var = 0
19
20     T_hat = c.T_hat(theta_hat, theta_check, pop_hat, pop_check, var)
21     T_check = c.T_check(theta_hat, theta_check, pop_hat, pop_check, var)
22
23     return theta_hat, theta_check, T_hat, T_check

```

Per disegnare i grafici calcolati nel Listato 12 si fa uso della libreria **matplotlib.pyplot** (vedi [6]).

## Listato 14: Initializer::showResultPlot

```

1  import matplotlib.pyplot as plt
2
3  ...
4
5  def saveResultVariation(self, variation_values, time_of_travel_hat, time_of_travel_check,
6      xLabel):
7      plt.plot(variation_values, time_of_travel_hat, label='Hat', color='blue')
8      plt.plot(variation_values, time_of_travel_check, label='Check', color='red')
9      plt.xlabel(xLabel)
10     plt.ylabel('Tempo di attraversamento della rete')
11     plt.title(f'Tempo di attraversamento della rete\nin funzione di: {xLabel}')
12     plt.legend()
13     plt.grid(True)
14     self.saveGraph(f'main_{xLabel}')
15
16 def saveResultThetaVariation(self, variation_values, thetas, pop, xLabel):
17     n = thetas.shape[0]
18
19     # Determinare dimensioni griglia
20     cols = int(np.ceil(np.sqrt(n)))
21     rows = int(np.ceil(n / cols))
22
23     fig, axes = plt.subplots(rows, cols, figsize=(4 * cols, 3 * rows))
24     axes = axes.flatten()
25
26     for i in range(n):
27         ax = axes[i]
28         ax.plot(variation_values, thetas[i, :], linewidth=1.8, color='red')
29
30         ax.set_title(f'Theta_{pop}_{i}')
31         ax.set_xlabel(xLabel)
32         ax.set_ylabel('Distribuzione popolazione')
33
34     # Usa la stessa scala su tutti i subplot
35     ax.set_ylim(0, 1)

```

```

35
36         ax.grid(True)
37
38         # Rimuovere eventuali subplot vuoti
39         for j in range(n, len(axes)):
40             fig.delaxes(axes[j])
41
42         plt.tight_layout()
43         self.saveGraph(f'{pop}_{xLabel}')

```

## 2.4 Classe Computer

Per calcolare gli equilibri di Nash e' necessario creare un'istanza della classe **Computer**. Tale operazione non deve, durante un utilizzo standard, essere effettuata dall'utente, bensì viene mascherata ad esso dall'utilizzo di **Initializer**, Listato 9

Listato 15: Computer

```

1  class Computer:
2      def __init__(self, param: AbstractParam):
3          self.param = param
4
5          #Parametro utile durante il calcolo di f_hat e f_check, vedi (*\autoref{script:12}*)
6          self.lmbda = 0.5*np.min([1/self.param.n_routes_hat, 1/self.param.n_routes_check])
7
8          #Array colonna di 1 con dimensioni n_routes_hat ed n_routes_check
9          #Utile durante il calcolo di f_hat e f_check, vedi (*\autoref{script:12}*)
10         self.one_n_hat = SafeArray(np.ones((self.param.n_routes_hat, 1)))
11         self.one_n_check = SafeArray(np.ones((self.param.n_routes_check, 1)))

```

Per calcolare l'equilibrio di Nash sfruttiamo la dimostrazione (vedi [3, Cap. 4]) del 1.4: individuiamo un punto fisso della funzione  $\mathcal{F}$ . Cominciamo da un  $(\hat{\theta}, \check{\theta}) \in S^{\hat{n}} \times S^{\check{\theta}}$  generico; si ricava  $\mathcal{F}(\hat{\theta}, \check{\theta})$ , che va a sostituire il valore di  $(\hat{\theta}, \check{\theta})$ , salvato invece come  $(\hat{\theta}_{prev}, \check{\theta}_{prev})$ .

Questa operazione viene ripetuta finché  $\|\hat{\theta} - \hat{\theta}_{prev}\| < \widehat{limit}$  e  $\|\check{\theta} - \check{\theta}_{prev}\| < \widetilde{limit}$ , dove i  $limit$  sono limiti di precisione definiti in **Initializer**, vedi Listato 9. Introduciamo anche un coefficiente di variazione e il numero di entità nelle popolazioni, utili per il calcolo effettuato nel Listato 13, che verrà applicato ai costi delle strade durante il calcolo dei tempi di percorrenza della rete, vedi Listato 20.

Listato 16: Computer::getNashEquilibriaVariation

```

1 def getNashEquilibriaVariation(self, theta_hat, theta_check, limit_hat, limit_check, zeta_hat,
  zeta_check, variationTime):
2     """
3     Ritorna l'Equilibrio di Nash variando i costi delle strade selezionate in param di un
      coefficiente variation, usando come
4     punto di partenza theta_hat, theta_check, come limite di precisione limit_hat,
      limit_check e come numero di entità zeta_hat e zeta_check
5     :param theta_hat, theta_check: vettore di dimensione [param.n_routes_hat, 1] e [param.
      n_routes_check, 1]
6     :param limit_hat, limit_check, variation, zeta_hat, zeta_check: float
7     """
8     prev_theta_hat = SafeArray(np.ones((self.param.n_routes_hat, 1))*np.inf)
9     prev_theta_check = SafeArray(np.ones((self.param.n_routes_check, 1))*np.inf)
10
11     count = 0
12
13     while np.linalg.norm(theta_hat-prev_theta_hat) > limit_hat or np.linalg.norm(theta_check-
      prev_theta_check) > limit_check:
14         prev_theta_hat[:] = theta_hat
15         prev_theta_check[:] = theta_check
16
17         theta_hat = self.f_hat(theta_hat, theta_check, zeta_hat, zeta_check, variationTime)
18         theta_check = self.f_check(theta_hat, theta_check, zeta_hat, zeta_check, variationTime
          )
19         count+=1
20
21     if self.param.show_iterations: print("Iterations: " + str(count))

```



```
22     return theta_hat, theta_check
```

Per calcolare l'equilibrio di Nash data unicamente una variazione sui tempi di percorrenza:

Listato 17: Computer::getNashEquilibriaCostVariation

```
1  def getNashEquilibriaCostVariation(self, theta_hat, theta_check, limit_hat, limit_check,
2      variation):
3      """
4          Ritorna l'Equilibrio di Nash variando i costi delle strade selezionate in param di
5          un coefficiente variation, usando come
6          punto di partenza theta_hat, theta_check, come limite di precisione limit_hat,
7          limit_check e come numero di entità i self.param.entity_number
8          :param theta_hat, theta_check: vettore di dimensione [param.n_routes_hat, 1] e [
9              param.n_routes_check, 1]
10         :param limit_hat, limit_check, variation, totPopulation: float
11         """
12         return self.getNashEquilibriaVariation(theta_hat,
13             theta_check,
14             limit_hat,
15             limit_check,
16             self.param.entity_number_hat,
17             self.param.entity_number_check,
18             variation)
```

Mentre per calcolarlo specificando una popolazione diversa da quella in **param**:

Listato 18: Computer::getNashEquilibriaCostVariation

```
1  def getNashEquilibriaPopVariation(self, theta_hat, theta_check, limit_hat, limit_check,
2      zeta_hat, zeta_check):
3      """
4          Ritorna l'Equilibrio di Nash usando come costi delle strade i tau delle rispettive
5          popolazioni presenti in param (senza variazioni), come
6          punto di partenza theta_hat, theta_check, come limite di precisione limit_hat,
7          limit_check e come numero di entità zeta_hat e zeta_check
```

```

5         :param theta_hat, theta_check: vettore di dimensione [param.n_routes_hat, 1] e [
           param.n_routes_check, 1]
6         :param limit_hat, limit_check, totPopulation, zeta_hat, zeta_check: float
7         """
8         return self.getNashEquilibriaVariation(theta_hat,
9                                                  theta_check,
10                                                 limit_hat,
11                                                 limit_check,
12                                                 zeta_hat,
13                                                 zeta_check,
14                                                 0)

```

Si può sfruttare la funzione **getNashEquilibria** per calcolare l'equilibrio di Nash senza introdurre una variazione ai costi delle strade:

Listato 19: Computer::getNashEquilibria

```

1  def getNashEquilibria(self, theta_hat, theta_check, limit_hat, limit_check):
2      """
3          Ritorna l'Equilibrio di Nash usando come costi delle strade i tau delle rispettive
           popolazioni presenti in param (senza variazioni e con numero di entità
4          nelle popolazioni 1), come punto di partenza theta_hat, theta_check e come limite
           di precisione limit_hat, limit_check
5          :param theta_hat, theta_check: vettore di dimensione [param.n_routes_hat, 1] e [
           param.n_routes_check, 1]
6          :param limit_hat, limit_check: float
7          """
8          return self.getNashEquilibriaVariation(theta_hat,
9                                                  theta_check,
10                                                 limit_hat,
11                                                 limit_check,
12                                                 self.param.entity_number_hat,
13                                                 self.param.entity_number_check,
14                                                 0)

```

Vengono introdotte le seguenti funzioni ausiliarie:

## Listato 20: Funzioni ausiliarie di Computer

```
1  #Equazione 1.5
2  def phi(self, x):
3      x = np.asarray(x)
4      res = np.empty_like(x)
5
6      mask = np.isinf(x)
7      res[mask] = 1.
8      res[~mask] = x[~mask] / (1. - x[~mask])
9
10     return SafeArray(res.reshape(-1, 1))
11
12 #Equazione 1.3 utilizzando i tau come nell'Equazione 1.2
13 def T_hat(self, theta_hat, theta_check, zeta_hat, zeta_check, variation):
14     nu_hat = self.param.Gamma_hat @ theta_hat
15
16     nu_check = self.param.Gamma_check @ theta_check
17
18     return self.param.Gamma_hat.T @ self.param.tau_hat_variated(nu_hat*zeta_hat, nu_check*
19         zeta_check, variation)
20
21 #Equazione 1.3 utilizzando i tau come nell'Equazione 1.2
22 def T_check(self, theta_hat, theta_check, zeta_hat, zeta_check, variation):
23     nu_hat = self.param.Gamma_hat @ theta_hat
24
25     nu_check = self.param.Gamma_check @ theta_check
26
27     return self.param.Gamma_check.T @ self.param.tau_check_variated(nu_hat*zeta_hat,
28         nu_check*zeta_check, variation)
29
30 #Equazione 1.6
31 def normalize(self, theta):
32     theta_max = np.maximum(theta, 0)
33     sum = np.sum(theta_max)
34     return theta_max / sum
```

Infine, vengono definite le funzioni per il calcolo di `f_hat` e `f_check` (vedi Equazione 1.8):

Listato 21: `Computer::f_hat`, `Computer::f_check`

```
1  def f_hat(self, theta_hat, theta_check, zeta_hat, zeta_check, variation):
2      composition = self.phi(self.T_hat(theta_hat, theta_check, zeta_hat, zeta_check,
3          variation))
4      return self.normalize(theta_hat - self.lmbda*(composition - (theta_hat.T @ composition
5          )[0]*self.one_n_hat))
6
7  def f_check(self, theta_hat, theta_check, zeta_hat, zeta_check, variation):
8      composition = self.phi(self.T_check(theta_hat, theta_check, zeta_hat, zeta_check,
9          variation))
10     return self.normalize(theta_check - self.lmbda*(composition - (theta_check.T @
11         composition)[0]*self.one_n_check))
```

## Riferimenti bibliografici

- 
- [1] D. Braess. Über ein Paradoxon aus der Verkehrsplanung. *Unternehmensforschung*, 12:258–268, 1968.
- [2] D. Braess. über ein Paradoxon aus der Verkehrsplanung. *Unternehmensforschung*, 12:258–268, 1968.
- [3] R. M. Colombo, L. Giuzzi, and F. Marcellini. Nash equilibria in traffic networks with multiple populations and origins-destinations. *J. Optim. Theory Appl.*, 206(2):Paper No. 31, 26, 2025.
- [4] R. M. Colombo and H. Holden. On the Braess paradox with nonlinear dynamics and control theory. *J. Optim. Theory Appl.*, 168(1):216–230, 2016.
- [5] R. M. Colombo, H. Holden, and F. Marcellini. On the microscopic modeling of vehicular traffic on general networks. *SIAM J. Appl. Math.*, 80(3):1377–1391, 2020.
- [6] matplotlib Developers. matplotlib user guide, 2026. 
- 
- [7] J. Nash. Non-cooperative games. *Ann. of Math. (2)*, 54:286–295, 1951.
- [8] J. F. Nash. *Non-Cooperative Games*. Phd thesis, Princeton University, Princeton, NJ, 1950.
- [9] NumPy Developers. Numpy user guide, 2026. 
- [10] Python Software Foundation. The python language reference, version 3.14, 2026.